KINEMAT — Constant-acceleration solver

TI-84 Evo / Evo-T · Python App
2026-07-26

Constant-acceleration motion has five quantities: v0, v, a, t and d. Tell the calculator which three you know and it returns the other two - so you never have to work out which equation to reach for.

WHAT IT ASKS YOU

1. Mode: 1 to 4, matching which three quantities you know
2. v0 (m/s) initial velocity - always asked
3. Then the other two knowns for that mode

WHAT IT SHOWS

- The two missing quantities, 3 decimals
- Units follow your inputs: SI gives m/s, m/s2, s and m

THE MATH

```
1 know v0,a,t: v = v0+at, d = v0t + at^2/2
2 know v0,v,t: a = (v-v0)/t, d = (v0+v)t/2
3 know v0,v,a: t = (v-v0)/a, d = (v^2-v0^2)/2a
4 know v0,a,d: v = sqrt(v0^2+2ad)
               t = (v-v0)/a
```

GET IT ONTO THE CALCULATOR

1. Open **connectevo.ti.com** in Chrome (v143+) and accept the terms.
2. Plug the calculator in with a USB-C *data* cable and switch it on.
3. Click **CONNECT TO CALCULATOR**, pick your calculator, click **Connect**.
4. Click **SEND TO CALCULATOR** and choose `KINEMAT.py`.
5. Check the name reads `KINEMAT`, not `PYTHON01`. Send.

RUN IT

`apps` → **Python App** → highlight `KINEMAT` → **Run**. Answer each prompt and press `enter`.

CHECK IT WORKS

```
KINEMATICS
1 v0,a,t  2 v0,v,t
3 v0,v,a  4 v0,a,d
Mode: 1
v0 (m/s): 0
a (m/s2): 9.81
t (s): 3
v = 29.430
d = 44.145
```

GOOD TO KNOW

- SI units in, SI units out. The program does no unit conversion, so do not mix km/h with m/s.
- NEGATIVES ARE MEANT TO BE USED: a = -5 for braking, a = -9.81 for downward gravity. They are not rejected.
- Modes 3 and 4 need a non-zero acceleration - at a = 0 the time is undefined. It re-asks rather than crashing.
- Mode 4 can be physically impossible: if v0^2 + 2ad < 0 it prints "No real answer" instead of a math domain error.
- d is DISPLACEMENT, not distance travelled. An object that goes up and comes back has d = 0 but travelled a longer path.
- Decimal comma works: 9,81 is accepted.
- For free fall from rest use mode 1 with v0 = 0 and a = 9.81.

> Mode does not matter. KINEMAT only does arithmetic and one square root - there is no trigonometry in it, so the DEGREE / RADIAN setting has no effect. (If you extend it to projectiles with launch angles, you would need radians() on the angle, because

SOURCE CODE — KINEMAT.PY

```
# KINEMAT -- constant-acceleration motion: know 3, get
the other 2
# TI-84 Evo / Evo-T / CE Python / 83 Premium CE Python
#
# Numeric output only. No CAS. Mode does not matter --
arithmetic only,
# no trig. SI units throughout: m/s, m/s^2, s, m.
# Negative values are legal: use them for deceleration or
downward motion.

from math import *


def getnum(msg):
    while True:
        s = input(msg)
        if "," in s and "(" not in s:
            s = s.replace(",", ".")
        try:
            return float(eval(s))
        except:
            print("Not a number")


def getnonzero(msg):
    while True:
        v = getnum(msg)
        if v != 0:
            return v
        print("Cannot be 0 here")


print("KINEMATICS")
print("1 v0,a,t  2 v0,v,t")
print("3 v0,v,a  4 v0,a,d")

while True:
    m = int(getnum("Mode: "))
    if m in (1, 2, 3, 4):
        break
    print("Pick 1 to 4")

v0 = getnum("v0 (m/s): ")

if m == 1:
    a = getnum("a (m/s2): ")
    t = getnum("t (s): ")
    print("v = {:.3f}".format(v0 + a * t))
    print("d = {:.3f}".format(v0 * t + a * t * t / 2))

elif m == 2:
    v = getnum("v (m/s): ")
    t = getnonzero("t (s): ")
    print("a = {:.3f}".format((v - v0) / t))
    print("d = {:.3f}".format((v0 + v) * t / 2))

elif m == 3:
    v = getnum("v (m/s): ")
    a = getnonzero("a (m/s2): ")
    print("t = {:.3f}".format((v - v0) / a))
    print("d = {:.3f}".format((v * v - v0 * v0) / (2 *
a)))

else:
    a = getnonzero("a (m/s2): ")
    d = getnum("d (m): ")
    q = v0 * v0 + 2 * a * d
    if q < 0:
        print("No real answer")
        print("v0^2+2ad < 0")
    else:
```

Python's math module is always in radians regardless of the MODE screen.)

```
        v = sqrt(q)
        print("v = {:.3f}".format(v))
        print("t = {:.3f}".format((v - v0) / a))
```